



**TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN**  
**KHOA TOÁN - CƠ - TIN HỌC**

**Tìm hiểu các phương pháp giải chính xác cho bài toán định tuyến  
phương tiện có ràng buộc khung thời gian (VRPTW)**

**Người thực hiện:**

| <b>Họ và tên</b> | <b>Mã sinh viên</b> |
|------------------|---------------------|
| Trần Đức Kiên    | 23000131            |
| Tạ Văn Nghĩa     | 23000145            |
| Vũ Tiến Đạt      | 23000111            |

Hà Nội, 25 - 4 - 2026

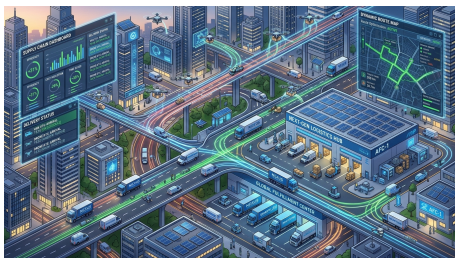
- 1 GIỚI THIỆU CHUNG
- 2 MÔ TẢ BÀI TOÁN
- 3 MÔ HÌNH HÓA BÀI TOÁN
- 4 BRANCH AND BOUND
- 5 BRANCH AND CUT
- 6 BRANCH AND PRICE
- 7 KẾT QUẢ THỰC NGHIỆM
- 8 KẾT LUẬN

# 1. GIỚI THIỆU CHUNG

# Bối cảnh và đặt vấn đề

## Bối cảnh

Trong bối cảnh ngành logistics và vận tải ngày càng phát triển, nhu cầu tối ưu hóa việc giao nhận hàng hóa trở nên đặc biệt quan trọng nhằm giảm chi phí vận hành, tiết kiệm thời gian và nâng cao chất lượng dịch vụ.



**Hình:** Tối ưu hóa logistics và vận tải trong hệ thống giao nhận hiện đại

# Bối cảnh và đặt vấn đề

## Đặt vấn đề

VRPTW không chỉ yêu cầu xác định lộ trình tối ưu cho các phương tiện mà còn phải đảm bảo mỗi khách hàng được phục vụ trong khoảng thời gian quy định.

Đây là bài toán thuộc lớp NP-hard, có độ phức tạp tính toán cao và mang nhiều ý nghĩa trong cả nghiên cứu lý thuyết lẫn ứng dụng thực tiễn.

## Lợi ích

Việc giải quyết bài toán VRPTW giúp:

- Giảm tổng quãng đường di chuyển
- Giảm chi phí nhiên liệu
- Tối ưu số lượng phương tiện sử dụng

## Mục tiêu bài báo cáo

- Tìm hiểu tổng quan bài toán VRPTW trong lĩnh vực logistics và vận tải.
- Xây dựng mô hình toán học gồm hàm mục tiêu, biến quyết định và các ràng buộc.
- Nghiên cứu các phương pháp giải chính xác như: Branch and Bound, Branch and Cut, Branch and Price.
- Thực nghiệm trên bộ dữ liệu Solomon Benchmark.
- Phân tích và đánh giá ưu điểm, hạn chế của các phương pháp giải chính xác.

## 2.MÔ TẢ BÀI TOÁN

# Tập hợp và tham số

## Tập hợp

- $V = \{0, 1, \dots, n\}$ : depot (0) và các khách hàng
- $K = \{1, 2, \dots, m\}$ : tập xe

## Tham số

- $q_i$ : nhu cầu khách hàng,  $Q$ : sức chứa xe
- $c_{ij}, t_{ij}$ : chi phí và thời gian di chuyển
- $[a_i, b_i]$ : khung thời gian phục vụ
- $s_i$ : thời gian phục vụ tại điểm  $i$

# Biến quyết định và hàm mục tiêu

## Biến quyết định

- $x_{ij} \in \{0, 1\}$ : đi từ  $i$  đến  $j$
- $s_i \geq 0$ : thời gian bắt đầu phục vụ
- $u_i$ : tải trọng tại mỗi điểm

## Hàm mục tiêu

$$\min Z = \sum_{i \in V} \sum_{\substack{j \in V \\ j \neq i}} c_{ij} x_{ij} \quad (1)$$

# Các ràng buộc chính

## Ràng buộc tuyến đường

- Mỗi khách hàng được phục vụ đúng 1 lần
- Bảo toàn luồng (xe đến điểm  $i$  thì phải rời khỏi điểm  $i$ )
- Số xe không vượt quá  $m$

## Ràng buộc thời gian và tải trọng

- $a_i \leq s_i \leq b_i$
- Ràng buộc thứ tự thời gian (Big-M)
- Giới hạn tải trọng  $Q$
- Ràng buộc nguyên

## Mô hình

Hàm mục tiêu:

$$\min Z = \sum_{i \in V} \sum_{j \in V, j \neq i} c_{ij} x_{ij} \quad (1)$$

Ràng buộc:

$$\sum_{j \in V, j \neq i} x_{ij} = 1 \quad \forall i \in V \setminus \{0\} \quad (2)$$

$$\sum_{i \in V, i \neq h} x_{ih} = \sum_{j \in V, j \neq h} x_{hj} \quad \forall h \in V \quad (3)$$

$$\sum_{j \in V \setminus \{0\}} x_{0j} \leq m \quad (4)$$

$$a_i \leq s_i \leq b_i \quad \forall i \in V \quad (5)$$

$$s_i + t_{ij} + s_i^{\text{service}} - s_j \leq M(1 - x_{ij}) \quad \forall i, j \in V \quad (6)$$

$$u_i + q_j - u_j \leq Q(1 - x_{ij}) \quad \forall i, j \in V \quad (7)$$

$$q_i \leq u_i \leq Q \quad \forall i \in V \setminus \{0\} \quad (8)$$

$$x_{ij} \in \{0, 1\} \quad \forall i, j \in V \quad (9)$$

## Ý nghĩa mô hình

- Đảm bảo tính khả thi của lộ trình
- Tối ưu chi phí vận chuyển
- Đáp ứng ràng buộc thời gian và năng lực xe

### 3.MÔ HÌNH HÓA BÀI TOÁN

# Mô hình hóa VRPTW bằng MILP

## Ý tưởng mô hình

VRPTW được mô hình hóa dưới dạng Quy hoạch Tuyến tính Nguyên hỗn hợp (MILP), kết hợp biến nhị phân chọn cung và các biến liên tục phụ trợ.

## Ba nhóm biến chính

- $x_{ij}$ : quyết định xe có đi trực tiếp từ  $i$  đến  $j$  hay không.
- $t_i$ : thời điểm xe rời khỏi nút  $i$ .
- $y_i$ : tải trọng trên xe sau khi rời khỏi nút  $i$ .

## Bản chất

Mô hình vừa chọn tuyến đường, vừa kiểm soát số xe, thời gian và sức chứa.

# Tập hợp và tham số

## Tập hợp

- $I = \{1, 2, \dots, n\}$ : tập khách hàng.
- 0: depot.
- $I_0 = I \cup \{0\}$ : tập tất cả các nút.

## Tham số chính

- $q_i$ : nhu cầu khách hàng  $i$ ,  $Q$ : sức chứa xe.
- $m$ : số lượng phương tiện tối đa có thể sử dụng.
- $[a_i, b_i]$ : cửa sổ thời gian phục vụ tại  $i$ .
- $\tau_{ij}$ : thời gian di chuyển từ  $i$  đến  $j$ , đã gồm thời gian phục vụ tại  $i$ .
- $c_{ij}$ : chi phí di chuyển trên cung  $(i, j)$ .

# Biến quyết định

## Biến nhị phân chọn cung

$$x_{ij} = \begin{cases} 1, & \text{nếu xe đi trực tiếp từ } i \text{ đến } j, \\ 0, & \text{ngược lại.} \end{cases}$$

## Biến liên tục phụ trợ

- $t_i \geq 0$ : thời điểm xe rời khỏi nút  $i$ .
- $y_i \geq 0$ : tải trọng trên xe sau khi rời khỏi nút  $i$ .

## Vai trò

$t_i$  kiểm soát ràng buộc thời gian, còn  $y_i$  kiểm soát ràng buộc sức chứa.

# Hàm mục tiêu

## Mục tiêu tối ưu

Tối thiểu hóa tổng chi phí di chuyển của toàn bộ các tuyến xe.

$$\min Z = \sum_{i \in I_0} \sum_{\substack{j \in I_0 \\ j \neq i}} c_{ij} x_{ij}$$

## Ý nghĩa

Cung  $(i, j)$  chỉ đóng góp vào chi phí khi  $x_{ij} = 1$ .

# Ràng buộc tuyến đường

## Phục vụ và bảo toàn luồng

Mỗi khách hàng được phục vụ đúng một lần:

$$\sum_{\substack{j \in I_0 \\ j \neq i}} x_{ij} = 1, \quad \forall i \in I$$

Xe đi vào khách hàng thì phải rời khỏi khách hàng đó:

$$\sum_{\substack{j \in I_0 \\ j \neq i}} x_{ij} - \sum_{\substack{j \in I_0 \\ j \neq i}} x_{ji} = 0, \quad \forall i \in I$$

## Giới hạn số xe

$$\sum_{j \in I} x_{0j} \leq m$$

## Cửa sổ thời gian

Mỗi khách hàng phải được phục vụ trong khoảng:

$$a_i \leq t_i \leq b_i, \quad \forall i \in I$$

## Liên kết thời gian trên cung

$$t_j \geq t_i + \tau_{ij} - T_{ij}(1 - x_{ij}), \quad \forall i, j \in I, i \neq j$$

$$T_{ij} = b_i + \tau_{ij} - a_j$$

## Liên kết tải trọng trên cung

$$y_j \geq y_i + q_j - Q_j(1 - x_{ij}), \quad \forall i, j \in I, i \neq j$$

$$Q_j = Q - q_j$$

## Giới hạn sức chứa

$$0 \leq y_i \leq Q, \quad \forall i \in I_0$$

## Mô hình tổng quát

$$\min Z = \sum_{i \in I_0} \sum_{\substack{j \in I_0 \\ j \neq i}} c_{ij} x_{ij} \quad (1)$$

$$\sum_{\substack{j \in I_0 \\ j \neq i}} x_{ij} = 1 \quad \forall i \in I \quad (\text{phục vụ một lần}) \quad (2)$$

$$\sum_{\substack{j \in I_0 \\ j \neq i}} x_{ij} - \sum_{\substack{j \in I_0 \\ j \neq i}} x_{ji} = 0 \quad \forall i \in I \quad (\text{bảo toàn luồng}) \quad (3)$$

$$\sum_{j \in I} x_{0j} \leq m \quad (\text{số xe sử dụng}) \quad (4)$$

$$t_j \geq t_i + \tau_{ij} - T_{ij}(1 - x_{ij}) \quad \forall i, j \in I, i \neq j \quad (\text{thời gian}) \quad (5)$$

$$y_j \geq y_i + q_j - Q_j(1 - x_{ij}) \quad \forall i, j \in I, i \neq j \quad (\text{tải trọng}) \quad (6)$$

$$a_i \leq t_i \leq b_i \quad \forall i \in I \quad (\text{cửa sổ thời gian}) \quad (7)$$

$$0 \leq y_i \leq Q \quad \forall i \in I_0 \quad (\text{sức chứa}) \quad (8)$$

$$x_{ij} \in \{0, 1\} \quad \forall i, j \in I_0, i \neq j \quad (\text{biến nhị phân}) \quad (9)$$

## 4. BRANCH AND BOUND

# Branch-and-Bound

## Khái quát

Branch-and-Bound là phương pháp giải chính xác cho các bài toán tối ưu tổ hợp và quy hoạch nguyên.

## Ba thao tác chính

- **Branching**: chia bài toán thành các bài toán con.
- **Bounding**: giải LP relaxation để tính cận dưới.
- **Pruning**: loại bỏ các nhánh không thể cải thiện nghiệm hiện tại.

## Trong VRPTW

Mỗi nút của cây tương ứng với một bài toán con của mô hình MILP.

# Ý tưởng Branch-and-Bound cho VRPTW

## Mô hình gốc

VRPTW được mô hình hóa bằng MILP với biến nhị phân:

$$x_{ij} \in \{0, 1\}$$

trong đó  $x_{ij} = 1$  nếu xe đi trực tiếp từ  $i$  đến  $j$ .

## LP Relaxation

Tại mỗi nút, thay ràng buộc nguyên bởi:

$$0 \leq x_{ij} \leq 1$$

Giá trị tối ưu của LP relaxation là cận dưới:

$$Z_{LP}(\omega)$$

# Cắt tỉa trong Branch-and-Bound

## Quy tắc cắt tỉa

Nút  $\omega$  bị loại khỏi stack  $\mathcal{S}$  nếu rơi vào một trong ba trường hợp:

- $LP(\omega)$  vô nghiệm — không tồn tại tuyến khả thi trong nhánh này.
- $LB(\omega) \geq Z^*$  — cận dưới không thể cải thiện nghiệm tốt nhất hiện tại.
- $\bar{x}_{ij} \in \{0, 1\} \forall i, j$  — nghiệm LP đã nguyên, đã dùng cập nhật  $Z^*$ .

## Ý nghĩa

Ba điều kiện trên đảm bảo loại bỏ sớm các nhánh không thể chứa nghiệm tối ưu mà không cần duyệt toàn bộ  $O(2^{n^2})$  nút.

# Chiến lược rẽ nhánh

## Chiến lược 1: Rẽ nhánh theo số xe

Tính tổng luồng rời depot trong nghiệm LP:

$$\bar{l} = \sum_{j \in I} \bar{x}_{0j}$$

Nếu  $\bar{l} \notin \mathbb{Z}$ , tạo hai nút con:

$$\omega_L = \omega \cup \left\{ \sum_{j \in I} x_{0j} \leq \lfloor \bar{l} \rfloor \right\}, \quad \omega_R = \omega \cup \left\{ \sum_{j \in I} x_{0j} \geq \lceil \bar{l} \rceil \right\}$$

## Mục đích

Cố định quy mô đội xe trước khi xử lý các biến cung, giúp loại bỏ sớm các cấu hình số xe không khả thi.

# Chiến lược rẽ nhánh

## Chiến lược 2: Rẽ nhánh theo biến cung

Nếu  $\bar{l} \in \mathbb{Z}$  nhưng vẫn còn biến cung phân số, chọn:

$$\bar{x}_{uv} = \arg \min_{0 < \bar{x}_{ij} < 1} |\bar{x}_{ij} - 0.5|$$

Tạo hai nút con:

$$\omega_L = \omega \cup \{x_{uv} = 0\}, \quad \omega_R = \omega \cup \{x_{uv} = 1\}$$

## Ý nghĩa

- $\omega_L$ : cấm cung  $(u, v)$  — xe không đi trực tiếp từ  $u$  đến  $v$ .
- $\omega_R$ : bắt buộc cung  $(u, v)$  — xe phải đi trực tiếp từ  $u$  đến  $v$ .
- Hai nhánh phân hoạch không gian nghiệm, đảm bảo không bỏ sót nghiệm tối ưu.

---

## Algorithm Khởi tạo thuật toán Branch-and-Bound

---

```
1: function BRANCH_AND_BOUND( $P_0, l, 0, c_{ij}, Q, [a_i, b_i], m$ )
2:    $S \leftarrow [\omega_0]$ 
3:    $Z^* \leftarrow +\infty$ 
4:    $S^* \leftarrow \emptyset$ 
5:   while  $S \neq \emptyset$  do
6:      $\omega \leftarrow \text{pop}(S)$ 
7:     ▷ Tiếp theo: giải LP và cắt tỉa
8:   end while
9:   return  $S^*, Z^*$ 
10: end function
```

---

## Algorithm Bounding và cắt tỉa tại nút $\omega$

```
1: function BOUNDING_AND_PRUNING( $LP(\omega), Z^*$ )
2:   Giải  $LP(\omega)$ : thay  $x_{ij} \in \{0, 1\}$  bằng  $0 \leq x_{ij} \leq 1$ 
3:   if  $LP(\omega)$  vô nghiệm then
4:     continue
5:   end if
6:    $(\bar{x}, \bar{t}, \bar{y}) \leftarrow$  nghiệm tối ưu của  $LP(\omega)$ 
7:    $LB(\omega) \leftarrow \sum_{i \in I_0} \sum_{j \neq i} c_{ij} \bar{x}_{ij}$ 
8:   if  $LB(\omega) \geq Z^*$  then
9:     continue
10:  end if
11:  if  $\bar{x}_{ij} \in \{0, 1\} \forall i, j$  then
12:    if  $\bar{x}$  tạo tuyến hợp lệ và  $LB(\omega) < Z^*$  then
13:       $Z^* \leftarrow LB(\omega)$ 
14:       $S^* \leftarrow (\bar{x}, \bar{t}, \bar{y})$ 
15:    end if
16:  continue
17: end if
18: end function
```

▷ Cắt tỉa: vô nghiệm

▷ Cắt tỉa: không cải thiện  $Z^*$

▷ Cắt tỉa: nghiệm đã nguyên

## Algorithm Rẽ nhánh tại nút $\omega$

```
1: function BRANCHING( $\omega, \bar{x}$ )
2:    $\bar{l} \leftarrow \sum_{j \in I} \bar{x}_{0j}$ 
3:   if  $\bar{l} \notin \mathbb{Z}$  then
4:     ▷ Rẽ nhánh theo số xe
5:      $\omega_L \leftarrow \omega \cup \{\sum_{j \in I} x_{0j} \leq \lfloor \bar{l} \rfloor\}$ 
6:      $\omega_R \leftarrow \omega \cup \{\sum_{j \in I} x_{0j} \geq \lceil \bar{l} \rceil\}$ 
7:   else
8:     ▷ Rẽ nhánh theo biến cung
9:      $\bar{x}_{uv} \leftarrow \arg \min_{0 < \bar{x}_{ij} < 1} |\bar{x}_{ij} - 0.5|$ 
10:     $\omega_L \leftarrow \omega \cup \{x_{uv} = 0\}$ 
11:     $\omega_R \leftarrow \omega \cup \{x_{uv} = 1\}$ 
12:  end if
13:  for  $\omega' \in \{\omega_R, \omega_L\}$  do
14:    if  $\omega'$  không vi phạm số xe, sức chứa, cửa sổ thời gian then
15:      push( $\mathcal{S}, \omega'$ )
16:    end if
17:  end for
18: end function
```

# Độ phức tạp thời gian

## Kích thước cây tìm kiếm

Với  $n$  khách hàng:  $O(n^2)$  biến nhị phân  $x_{ij} \Rightarrow$  cây B&B có tối đa  $O(2^{n^2})$  nút.

## Chi phí mỗi nút

Mỗi nút giải một LP kích thước  $O(n^2)$ , chi phí  $T_{LP}(n) = O(n^{4.5})$ .

## Trường hợp xấu nhất

$$T_{\text{total}} = O(2^{n^2} \cdot n^{4.5})$$

## Thực tế

Cắt tỉa theo  $LB(\omega) \geq Z^*$  loại bỏ phần lớn các nhánh:

$$N_{\text{visited}} \ll 2^{n^2}$$

# Độ phức tạp không gian

## Mỗi nút

Lưu tập ràng buộc bổ sung tích lũy từ gốc  $\Rightarrow O(n^2)$  bộ nhớ.

## Toàn bộ stack DFS

Stack  $S$  chứa tối đa  $O(n^2)$  nút tại mỗi thời điểm (bằng độ sâu cây), do đó:

$$S_{\text{DFS}} = O(n^2) \times O(n^2) = O(n^4)$$

## So sánh với Best-First Search

Best-first phải lưu toàn bộ frontier đồng thời  $\Rightarrow$  có thể lên tới  $O(2^{n^2})$  nút trong trường hợp xấu nhất. DFS tiết kiệm bộ nhớ hơn đáng kể.

## 5. BRANCH AND CUT

# Khái quát Branch-and-Cut

- Branch-and-Cut là phiên bản nâng cao của Branch-and-Bound, kết hợp giữa:
  - **Branch-and-Bound** (phân nhánh và cận)
  - **Cutting Planes** (mặt cắt)
- Ý tưởng chính:
  - Không chỉ phân nhánh khi gặp nghiệm phân số
  - Chủ động thêm các *valid inequalities*
- Mục tiêu:
  - Thắt chặt LP relaxation
  - Cải thiện lower bound
  - Giảm số node cần duyệt

**Valid Inequality** là các ràng buộc được thêm vào sau bước nới lỏng tuyến tính (LP Relaxation) nhằm thu nhỏ không gian nghiệm bằng cách loại bỏ một phần nghiệm không nguyên. Ở đây ta sẽ sử dụng ràng buộc về sức chứa để làm bất đẳng thức hợp lệ.

Capacity inequality:

$$\sum_{(i,j) \in A: i \in S, j \notin S} x_{ij} \geq k(S)$$

trong đó:

$$k(S) = \left\lceil \frac{\sum_{i \in S} q_i}{Q} \right\rceil$$

là số xe tối thiểu cần để phục vụ tập S

# Ví dụ Capacity Cut

Giả sử đội xe có  $Q = 20$ . Xét tập khách hàng  $S = \{1, 2, 3\}$  với nhu cầu tương ứng là  $q_1 = 8$ ,  $q_2 = 7$ , và  $q_3 = 10$ .

Tổng nhu cầu của tập  $S$  là  $\sum_{i \in S} q_i = 25$ .

Số xe tối thiểu:

$$k(S) = \left\lceil \frac{25}{20} \right\rceil = 2$$

Tổng luồng xe hiện tại:

$$f(S) = \sum_{i \in \{1,2,3\}, j \notin \{1,2,3\}} \hat{x}_{ij}$$

Nếu  $f(S) < k(S)$  thì thêm Capacity cut:

$$\sum_{i \in \{1,2,3\}, j \notin \{1,2,3\}} x_{ij} \geq 2$$

Ý nghĩa ràng buộc: thiết lập một mặt cắt bao quanh tập  $S$ , ép tổng trọng số của tất cả các cung có hướng đi ra khỏi  $S$  phải lớn hơn hoặc bằng 2.

---

## Algorithm Hàm kiểm tra và thêm ràng buộc về sức chứa

---

```
1: function ADD_CAPACITY_CONSTRAINT( $LP(\omega)$ ,  $S$ ,  $\hat{x}$ )
2:    $sum \leftarrow 0$ 
3:   for  $i \in S$  do
4:      $sum \leftarrow sum + q_i$ 
5:   end for
6:    $k(S) \leftarrow \lceil sum/Q \rceil$ 
7:
```

$$f(S) \leftarrow \sum_{i \in S, j \notin S} \hat{x}_{ij}$$

```
8:   if  $f(S) < k(S)$  then
9:     Add:
```

$$\sum_{i \in S, j \notin S} x_{ij} \geq k(S)$$

```
10:  end if
11: end function
```

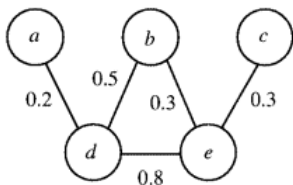
---

# Graph Shrinking Heuristic

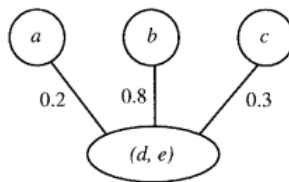
Ta sử dụng thuật toán Heuristic **Graph Shrinking** để tìm và xác định các tập  $S$  vi phạm ràng buộc về sức chứa, Cấu trúc dữ liệu Union-Find được áp dụng để triển khai thuật toán

Ý tưởng:

- Gộp các customer có luồng phân số lớn
- Tạo thành *supernode*
- Kiểm tra  $f(S) < k(S)$  và thêm ràng buộc bằng `add_capacity_constraint()`



(a) Before shrinking



(b) After shrinking  $d$  and  $e$

Hình: Quá trình gộp nút

# Ví dụ Graph Shrinking

Giả sử LP relaxation cho kết quả:

$$\hat{x}_{12} = 0.92, \quad \hat{x}_{23} = 0.88, \quad \hat{x}_{31} = 0.75, \quad \hat{x}_{45} = 0.31$$

Ban đầu, ta khởi tạo các supernode độc lập cho mỗi nút:

$$S_1 = \{1\}, \quad S_2 = \{2\}, \quad S_3 = \{3\}, \quad S_4 = \{4\}, \quad S_5 = \{5\}$$

Danh sách cung  $L$  được sắp xếp theo  $\hat{x}_{ij}$  giảm dần:

$$L = \{(1, 2), (2, 3), (3, 1), (4, 5)\}$$

Thuật toán chọn ngẫu nhiên một cung trong  $\mu$  cung lớn nhất (ở ví dụ này ta xét thứ tự giảm dần trên  $L$ ).

**Quá trình gộp nút:**

- **Chọn cung (1, 2):** Gộp thành siêu nút  $S_{12} = \{1, 2\}$ . Gọi `add_capacity_constraint()`
- **Chọn cung (2, 3):** Gộp tiếp nút 3 vào tạo thành siêu nút  $S_{123} = \{1, 2, 3\}$ . Gọi `add_capacity_constraint()`.
- **Chọn cung (3, 1):** Do  $S_1 = S_{123} = S_3$ . nên ta bỏ qua

Thuật toán dừng một vòng lặp khi danh sách  $L$  rỗng hoặc không còn cung nào

# Thuật toán Graph Shrinking

## Algorithm Hàm graph Shrinking

```
1: function GRAPH_SHRINKING( $\omega, \hat{x}, \mu, Q$ )
2:    $A_0 \leftarrow \{(i, j) \in A \mid i \neq 0 \text{ và } j \neq 0 \text{ và } \hat{x}_{ij} > \epsilon\}$ 
3:   for iter = 1 to MAX_ITER do
4:     MAKESET( $\forall v \in N$ ) ▷ Khởi tạo:  $\text{parent}[v] \leftarrow v, q(v) \leftarrow q_v, S_v \leftarrow \{v\}$ 
5:      $L \leftarrow \text{sort}(A_0 \text{ theo } \hat{x}_{ij} \text{ giảm dần})$ 
6:     while  $L \neq \emptyset$  do
7:       Chọn ngẫu nhiên cung  $(u, v)$  trong  $\mu$  cung đầu tiên của  $L$ 
8:        $L \leftarrow L \setminus \{(u, v)\}$ 
9:        $r_u \leftarrow \text{FIND}(u)$ ,  $r_v \leftarrow \text{FIND}(v)$ 
10:      if  $r_u \neq r_v$  then
11:        UNION( $r_u, r_v$ ) ▷ Gộp hai tập hợp
12:         $r_{\text{new}} \leftarrow \text{FIND}(u)$ 
13:         $S_{\text{new}} \leftarrow \text{Tập hợp các nút thuộc nhánh } r_{\text{new}}$ 
14:        ADD_CAPACITY_CONSTRAINT( $\text{LP}(\omega), S_{\text{new}}, \hat{x}$ )
15:      end if
16:    end while
17:  end for
18: end function
```

# Độ phức tạp Graph Shrinking

Trong mỗi lần lặp:

$$O(n \cdot |A| \cdot \log |A|)$$

Đặt  $MAX\_ITER$  là số vòng lặp tối đa

Tổng độ phức tạp:

$$O(MAX\_ITER \cdot n \cdot |A| \cdot \log |A|)$$

Trường hợp đặc biệt:

- Đồ thị thưa:

$$O(n^2 \log n)$$

- Đồ thị dày:

$$O(n^3 \log n)$$

# Branch-and-Cut Framework

Branch-and-Cut giữ nguyên toàn bộ khung duyệt cây của Branch-and-Bound. Điểm khác biệt duy nhất là sau khi giải LP relaxation tại mỗi nút, ta chèn thêm một **vòng lặp tìm mặt cắt** (cutting plane loop) trước khi quyết định phân nhánh.

Lợi ích:

- Thắt chặt lower bound sớm
- Giảm mạnh số node
- Cắt tĩa hiệu quả hơn

---

## Algorithm Vòng lặp Cutting Plane

---

```
1: // Chèn vào sau bước giải LP Relaxation của Branch-and-Bound
2: repeat
3:   GRAPHSHRINKING( $\omega, \bar{x}, \mu$ )
4:   Resolve LP( $\omega$ )
5:   if LP infeasible then
6:     break
7:   end if
8:   Update lower bound
9:   if  $LB(\omega) \geq Z^*$  then
10:    break
11:  end if
12: until No violated cuts found
13: // Tiếp tục bước kiểm tra tính nguyên hoặc phân nhánh của Branch-and-Bound
```

---

# Độ phức tạp Branch-and-Cut

Worst-case search tree:

$$O(2^{n^2})$$

Chi phí tại mỗi node:

$$O(p(\omega) \cdot n^{4.5})$$

trong đó:

- $p(\omega)$ : số lần cutting-plane loop

Tổng độ phức tạp:

$$O(2^{n^2} \cdot \bar{p} \cdot n^{4.5})$$

Trade-off:

- Tổn thêm thời gian giải LP
- Nhưng giảm mạnh số node duyệt

## 6.BRANCH AND PRICE

# Branch and Price – Khái quát

**Branch and Price** kết hợp hai kỹ thuật:

- **Branch and Bound** – duyệt cây tìm kiếm, cắt tỉa nhánh không tối ưu nhờ *hàm đánh giá cận*.
- **Column Generation** – tại mỗi nút chỉ sinh tuyến đường *thực sự có ích*, không liệt kê toàn bộ.

## Tại sao cần Column Generation?

VRPTW với  $n$  khách hàng: số tuyến khả thi  $\sim 2^n$

⇒ Không liệt kê hết

⇒ Duy trì tập con trong **RMP**, bổ sung dần qua SPPRC

## Cài đặt thực tế (R101 – 100 KH):

- ParamsVRP: quản lý dist, ttime, a, b, d, s
- ColumnGeneration: giải RMP bằng Gurobi
- SPPRC: bài toán con Label Setting
- BranchAndBound: điều phối cây TreeBB
- Route: biểu diễn tuyến đường + route.Q

# Khởi tạo – Dummy Routes và ma trận dist

## Depot nhân đôi

- Đỉnh 0: depot xuất phát
- Đỉnh  $N-1$ : depot kết thúc (cùng tọa độ)
- SPPRC tìm đường từ  $0 \rightarrow N-1$

## Ma trận dist – ngăn chu trình sai

- $\text{dist}[i][0] = \infty \forall i$ : cấm về depot đầu
- $\text{dist}[N-1][j] = \infty \forall j$ : cấm rời depot cuối
- $\text{dist}[i][i] = \infty \forall i$ : cấm tự vòng

## Dummy routes (R101 – 100 KH)

Mỗi khách hàng  $i$  một xe riêng:

$$\text{route}_i = [0, i, N-1]$$

$$c_i = \text{dist}[0][i] + \text{dist}[i][N-1]$$

100 dummy routes  $\Rightarrow$  RMP

luôn có nghiệm khả thi ban đầu.

$\text{route.Q} = 1.0$  cho tất cả routes khởi tạo.

# Column Generation – Restricted Master Problem (RMP)

## Mô hình Set Covering LP

$$\min \sum_{r \in R} c_r y_r$$

$$\sum_{r \in R} a_{ir} y_r \geq 1 \quad \forall i \in \{1, \dots, N-2\}$$

$$y_r \geq 0 \quad \forall r \in R$$

- $c_r$ : tổng khoảng cách Euclid của tuyến  $r$
- $a_{ir} = 1$  nếu KH  $i$  nằm trong tuyến  $r$
- $y_r$  thực (nới lỏng từ nhị phân)
- Sau hội tụ:  $\text{route.Q} \leftarrow y_r$   
dùng trong B&B kiểm tra tính nguyên

## Vòng lặp Column Generation

❶ Giải RMP (Gurobi)  $\Rightarrow Z_{LP}$

❷ Lấy dual:

$$\pi_i \leftarrow \text{constraints}[i].Pi$$

❸ Cập nhật reduced cost:

$$\text{cost}[i][j] \leftarrow \text{dist}[i][j] - \pi_i$$

(depot 0 và  $N-1$  **không** trừ  $\pi_i$ )

❹ Gọi SPPRC tìm tuyến  $\bar{c}_r < 0$

❺ Không còn tuyến nào  $\Rightarrow$  **hội tụ**

❻ Thêm tuyến vào RMP  
*incrementally*

# Bài toán con SPPRC – Label Setting

**Bài toán:** Tìm tuyến  $0 \rightarrow N-1$  thỏa mãn đồng thời:

- Sức chứa:  $\sum d_i \leq Q$  | Time window:  $\text{ttime} \leq b_i$ ; đến sớm hơn  $a_i$  thì chờ
- Không lặp đỉnh | Reduced cost  $\bar{c}_r < 0$

**Nhãn  $L$  – biểu diễn đường đi khả thi từ depot đến đỉnh hiện tại:**

$$L = (\text{city}, \text{cost}, \text{ttime}, \text{demand}, \text{vis})$$

- city: đỉnh hiện tại | cost: tổng reduced cost tích lũy
- ttime: thời gian thực tế đến city (bao gồm thời gian chờ và phục vụ)
- demand: tổng nhu cầu tích lũy | vis: mảng boolean đánh dấu đỉnh đã thăm

**Hàm mở rộng (REF): từ đỉnh  $u$  sang đỉnh  $v$**

$$\text{tt}' = \max(a_v, \text{ttime} + t_{uv} + s_u), \quad d' = \text{demand} + d_v, \quad \text{cost}' = \text{cost} + \text{cost}[u][v]$$

Chấp nhận nhãn mới nếu:  $\text{tt}' \leq b_v$  và  $d' \leq Q$

**Mục đích:** Loại bỏ nhãn kém – nhãn bị dominate không thể dẫn đến nghiệm tốt hơn, loại sớm để giảm không gian tìm kiếm.

$L_1$  dominate  $L_2$  tại cùng đỉnh nếu đồng thời:

$$L_1.\text{cost} \leq L_2.\text{cost} \quad L_1.\text{ttime} \leq L_2.\text{ttime} \quad L_1.\text{demand} \leq L_2.\text{demand}$$

$$\forall k: L_1.\text{vis}[k] = \text{True} \Rightarrow L_2.\text{vis}[k] = \text{True}$$

$\Rightarrow$  Loại  $L_2$  khỏi tập  $U$  (unprocessed labels)

**Ý nghĩa điều kiện vis:**  $L_1$  không bị hạn chế mở rộng hơn  $L_2$  – mọi đỉnh  $L_2$  còn có thể thăm thì  $L_1$  cũng có thể thăm, trong khi  $L_1$  tốt hơn hoặc bằng  $L_2$  trên cả 3 tài nguyên còn lại.

Dominance được kiểm tra theo cặp trong `city2labels[L.city]` – danh sách nhãn tại mỗi đỉnh – giúp giảm đáng kể số nhãn cần xử lý, đặc biệt hiệu quả với các bộ dữ liệu có time window hẹp.

## Kỹ thuật tăng tốc Feillet 2004:

Ngay sau khi mở rộng đến đỉnh  $v$ , kiểm tra trước từng đỉnh  $j$  chưa thăm. Nếu từ  $v$  đi thẳng đến  $j$  đã vi phạm thì khóa  $j$  luôn:

$$tt' + t_{vj} + s_v > b_j \quad \text{hoặc} \quad d' + d_j > Q \quad \Rightarrow \quad vis'[j] \leftarrow \text{True}$$

Ví dụ *R101*: KH 59 có TW [18, 28] rất hẹp, demand = 28  $\Rightarrow$  khóa trước hầu hết KH tiếp theo ngay từ bước đầu.

## Thu thập nghiệm:

- Nhãn đến depot cuối  $N-1$  có  $\text{cost} < -10^{-4} \Rightarrow$  đưa vào tập  $P$
- Truy vết ngược qua `index_prev_label` để tái tạo tuyến đường
- Gọi `switch_path()` để đảo ngược thứ tự: depot đầu  $\rightarrow \dots \rightarrow$  depot cuối
- Trả tối đa  $\text{maxsol} = 2(N-1)$  tuyến có reduced cost âm về RMP

# Hàm đánh giá cận – Định nghĩa và Tính hợp lệ

## Định nghĩa (lý thuyết nhánh cận):

Tại mỗi lời giải bộ phận (nút trong cây B&B), cần hàm  $g$  thỏa mãn:

$$g \leq \min \{ f(x) : x \text{ là nghiệm nguyên mở rộng từ nút này} \}$$

Nếu  $g > f^* = \text{UB} \Rightarrow$  **Cắt tỉa** toàn bộ cây con – không thể tốt hơn  $f^*$

**Trong Branch and Price:**  $g = Z_{LP}$  từ Column Generation

$$g(\text{nút}) = Z_{LP} = \min_{y_r \geq 0} \sum_{r \in R} c_r y_r \quad \text{s.t. ràng buộc phủ KH} + \text{ràng buộc phân nhánh}$$

Tại sao  $Z_{LP}$  là cận dưới hợp lệ? – 3 lý do:

### 1. LP Relaxation $\leq$ ILP

Mọi nghiệm nguyên khả thi đều là nghiệm LP khả thi  $\Rightarrow Z_{LP} \leq Z_{ILP}$

### 2. Column Generation cho $Z_{LP}$ chính xác

SPPRC xác nhận không còn cột nào có reduced cost âm  $\Rightarrow$  RMP đạt tối ưu LP toàn phần

### 3. Ràng buộc phân nhánh được nhúng trực tiếp vào dist

SPPRC chỉ sinh tuyến có  $\text{dist}[u][v] < \infty \Rightarrow$  mọi tuyến hợp lệ với nhánh hiện tại

# Hàm đánh giá cận – Điều kiện chặt nhánh

## Điều kiện chặt nhánh (Bound cut):

$$\underbrace{Z_{LP}}_{g(\text{nút})} > \underbrace{UB}_{f^*} \Rightarrow \text{Mọi nghiệm nguyên trong cây con} \geq Z_{LP} > f^*$$

$\Rightarrow$  Không thể tìm nghiệm tốt hơn  $f^*$   $\Rightarrow$  Cắt tỉa, không xét tiếp

## Ưu điểm của $Z_{LP}$ so với Branch and Bound thuần túy:

- B&B thuần túy: cận dưới yếu (thường là 0 hoặc heuristic đơn giản)
- Branch and Price:  $Z_{LP}$  từ Column Generation rất sát  $Z_{ILP}$   
 $\Rightarrow$  Bound cut hiệu quả ngay từ sớm, cắt bỏ phần lớn cây tìm kiếm
- Đặc biệt hiệu quả với VRPTW nhờ cấu trúc Set Covering có tính chất LP tốt

## Tóm tắt vai trò các thành phần trong hàm đánh giá cận:

| Thành phần   | Vai trò               | Đảm bảo                        |
|--------------|-----------------------|--------------------------------|
| Gurobi (RMP) | Giải LP               | $Z_{LP}$ chính xác             |
| SPPRC        | Kiểm tra tối ưu LP    | Không còn cột cải thiện        |
| dist chỉnh   | Nhúng ràng buộc nhánh | Cận dưới đúng với nút hiện tại |

# Phân nhánh theo cạnh – Ma trận edges

Sau khi **Column Generation** hội tụ, kiểm tra tính nguyên của nghiệm LP:

$$\text{edges}[i][j] = \sum_{r \in R} \text{route.Q}[r] \cdot \mathbf{1}[(i, j) \in r]$$

Trong đó  $\text{route.Q}[r] = y_r$  – giá trị LP của tuyến  $r$  sau khi RMP hội tụ.

Nếu  $\exists(i, j): 0 < \text{edges}[i][j] < 1 \Rightarrow$  nghiệm phân số  $\Rightarrow$  cần phân nhánh

**Tiêu chí chọn cạnh phân nhánh  $(i^*, j^*)$ :**

$$(i^*, j^*) = \arg \max_{0 < e_{ij} < 1} \min(e_{ij}, |1 - e_{ij}|) \cdot c_r$$

- $\min(e_{ij}, |1 - e_{ij}|)$ : ưu tiên cạnh **xa 0 và 1 nhất** (gần 0.5)  
 $\Rightarrow$  Hai nhánh con cân bằng hơn, tránh phân nhánh lệch
- $\times c_r$ : ưu tiên cạnh thuộc tuyến **chi phí lớn nhất**  
 $\Rightarrow$  Tác động lớn hơn đến hàm mục tiêu khi phân nhánh
- $\text{best\_val} = 0$  nếu  $e_{ij} < 0.5$ ;  $\text{best\_val} = 1$  nếu  $e_{ij} \geq 0.5$

# Phân nhánh theo cạnh – Áp đặt lên dist

**Áp đặt ràng buộc phân nhánh trực tiếp lên ma trận dist:**

*Nhánh cấm* ( $val=0$ ):  $dist[i^*][j^*] \leftarrow \infty$

*Nhánh bắt buộc* ( $val=1$ ):

- Cấm tất cả cạnh ra khỏi  $i^*$ , chỉ giữ  $(i^*, j^*)$ :  $dist[i^*][:] \leftarrow \infty$ ;  
 $dist[i^*][j^*] \leftarrow dist\_base[i^*][j^*]$
- Cấm tất cả cạnh vào  $j^*$ , chỉ giữ  $(i^*, j^*)$ :  $dist[:,j^*] \leftarrow \infty$ ;  
 $dist[i^*][j^*] \leftarrow dist\_base[i^*][j^*]$
- Cấm cạnh ngược  $(j^*, i^*)$ :  $dist[j^*][i^*] \leftarrow \infty$
- *Ngoại lệ*: depot 0 và  $N-1$  không bị cấm toàn bộ

SPPRC tự loại tuyến vi phạm vì  $dist = \infty \Rightarrow$  không cần thêm ràng buộc vào LP

**Cơ chế đệ quy (recur=True):**

- 1 Phục hồi  $dist \leftarrow copy(dist\_base)$
- 2 Gọi đệ quy `edges_based_on_branching(..., recur=True)`  
 $\Rightarrow$  Tái áp dụng **toàn bộ chuỗi** ràng buộc từ gốc đến nút hiện tại
- 3 Đảm bảo tính nhất quán của  $dist$  tại mọi nút trong cây

# Cập nhật LB, UB và Điều kiện cắt tỉa

## Cận trên UB – $f^*$

Khởi tạo:  $UB = +\infty$

Cập nhật khi nghiệm nguyên  
( $edges[i][j] \in \{0, 1\} \forall i, j$ ):

$$UB \leftarrow Z_{LP} = f^*$$

$best\_routes \leftarrow \{r : route.Q > 10^{-6}\}$

## Cận dưới LB

Tại nút `toplevel` (DFS cấp cao nhất), sau khi cả hai nhánh con xử lý xong:

$$LB = \min(son0.lowest, son1.lowest)$$

$Z_{LP}$  tại mỗi nút là cận dưới cục bộ của toàn bộ cây con bên dưới.

## 3 điều kiện cắt tỉa

### 1. Gap đủ nhỏ:

$$\frac{UB - LB}{UB} < \varepsilon = 10^{-6}$$

$\Rightarrow$  Kết thúc toàn bộ thuật toán

### 2. Relaxation vô nghiệm:

$$Z_{LP} > 2 \cdot \text{maxlength} \text{ hoặc } Z_{LP} < 0$$

$\Rightarrow$  Nhánh LP vô nghiệm, cắt tỉa

### 3. Bound cut – điều kiện chặt nhánh chính:

$$\underbrace{Z_{LP}}_{g(\text{nút})} > \underbrace{UB}_{f^*}$$

$\Rightarrow$  Cây con không thể cho nghiệm tốt hơn  $f^*$

$\Rightarrow$  Cắt tỉa, không xét tiếp

# Độ phức tạp

## Từng thành phần

- **SPPRC** (worst case):  
 $O(2^n \cdot n^2)$  do không gian visited  
→ Dominance + Feillet 2004 cắt giảm mạnh
- **Column Generation / nút:**  
 $O(n \cdot |R|^{1.5})$  mỗi vòng lặp (Simplex)
- **Cây B&B:**  
Tối đa  $O(2^{n^2})$  nút theo số biến  $x_{ij}$   
→ Bound cut từ  $Z_{LP}$  tốt cắt bỏ phần lớn

## Độ phức tạp tổng thể (worst case)

$$T(n) = O(2^{n^2} \cdot 2^n \cdot n^4)$$

## Thực nghiệm – Solomon Benchmark

Bộ R, C, RC với 25–100 khách hàng.  
Thuật toán hội tụ nhanh nhờ:

- $Z_{LP}$  sát  $Z_{ILP} \Rightarrow$  bound cut hiệu quả
- Dominance giảm số nhãn SPPRC
- Feillet 2004 khóa trước đỉnh không khả thi

## 7.KẾT QUẢ THỰC NGHIỆM

# Thiết lập thực nghiệm

## Thuật toán khảo sát

- Branch and Bound (B&B)
- Branch and Cut (B&C)
- Branch and Price (B&P)

## Quy mô bài toán ( $n$ )

Thực nghiệm trên số lượng khách hàng :

$$n \in \{5, 8, 11\}$$

## Bộ dữ liệu chuẩn (Solomon)

**C101:** Dữ liệu phân cụm  
(*Clustered*)

**R101:** Dữ liệu ngẫu nhiên  
(*Random*)

**RC101:** Dữ liệu hỗn hợp  
(*Random-Clustered*)

*Tổng cộng có 27 kịch bản thực nghiệm khác nhau ( $3 \text{ thuật toán} \times 3 \text{ bộ dữ liệu} \times 3 \text{ mức quy mô}$ ).*

# Hệ thống 10 tiêu chí đánh giá

## Chất lượng nghiệm

- **Obj** — Chi phí khoảng cách nhỏ nhất
- **Gap (%)** —  $\frac{|Obj - LB|}{Obj} \times 100\%$   
0.00%  $\Rightarrow$  nghiệm tối ưu toàn cục

## Hiệu năng tính toán

- **Time (s)** — Thời gian đến khi hội tụ
- **T.Node (ms/node)** — Thời gian trung bình/nút

## Cấu trúc cây tìm kiếm

- **Nodes** — Số nút (LP Relaxation)
- **Branches** — Số lần phân nhánh

## Hiệu quả cắt tỉa

- **Pruned** — Số nút bị loại (vi phạm cận)
- **P.Rate (%)** — Tỷ lệ nút bị cắt tỉa

## Thông tin nút

- **Inf.** — Số nút LP vô nghiệm
- **Int.** — Số nghiệm nguyên tìm được

## Mục tiêu

Gap = 0.00% với thời gian nhỏ nhất

# C101 — Phân cụm: Kết quả tổng hợp

| Tiêu chí    | $n = 5$ |       |       | $n = 8$ |       |       | $n = 11$ |       |        |
|-------------|---------|-------|-------|---------|-------|-------|----------|-------|--------|
|             | B&B     | B&C   | B&P   | B&B     | B&C   | B&P   | B&B      | B&C   | B&P    |
| Obj         | 42.42   | 42.42 | 42.41 | 49.72   | 49.72 | 49.72 | 59.49    | 59.49 | 59.49  |
| Gap (%)     | 0%      | 0%    | 0%    | 0%      | 0%    | 0%    | 0%       | 0%    | 0%     |
| Time (s)    | 0.03    | 0.03  | 0.01  | 1.03    | 0.13  | 0.07  | 473.92   | 77.80 | 0.50   |
| T.Node (ms) | 6.96    | 6.95  | 11.02 | 12.12   | 14.44 | 70.00 | 23.09    | 33.75 | 500.00 |
| Nodes       | 5       | 5     | 1     | 85      | 9     | 1     | 20521    | 2305  | 1      |
| Branches    | 2       | 2     | 0     | 84      | 8     | 0     | 20520    | 2304  | 0      |
| P.Rate (%)  | 0%      | 0%    | 100%  | 49.4%   | 43.8% | 100%  | 50.0%    | 49.9% | 100%   |
| Inf.        | 2       | 2     | N/A   | 28      | 4     | N/A   | 7122     | 864   | N/A    |
| Int.        | 1       | 1     | N/A   | 4       | 2     | N/A   | 12       | 5     | N/A    |

**C101** ( $n = 11$ ): B&P nhanh hơn B&B  $\approx 948\times$ , nhanh hơn B&C  $\approx 156\times$

# C101 — So sánh thời gian chạy



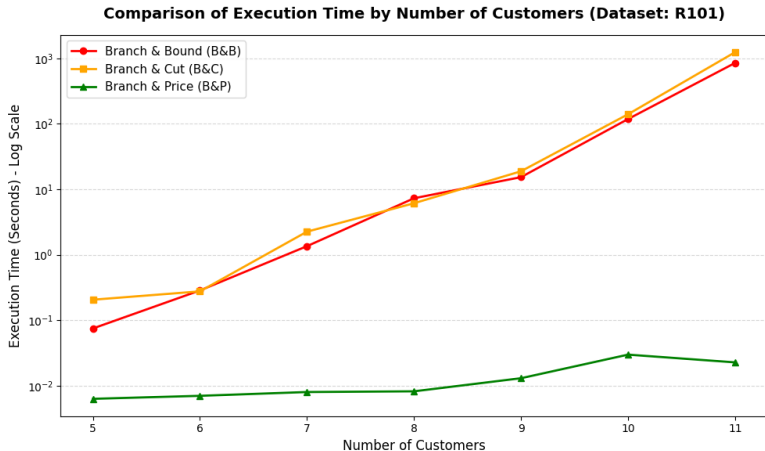
\* Trục Y dùng thang logarithm để hiển thị rõ sự chênh lệch

# R101 — Ngẫu nhiên: Kết quả tổng hợp

| Tiêu chí   | $n = 5$ |        |        | $n = 8$ |        |        | $n = 11$ |        |        |
|------------|---------|--------|--------|---------|--------|--------|----------|--------|--------|
|            | B&B     | B&C    | B&P    | B&B     | B&C    | B&P    | B&B      | B&C    | B&P    |
| Obj        | 156.35  | 156.35 | 156.35 | 216.84  | 216.84 | 216.84 | 297.79   | 297.79 | 297.79 |
| Gap (%)    | 0%      | 0%     | 0%     | 0%      | 0%     | 0%     | 0%       | 0%     | 0%     |
| Time (s)   | 0.07    | 0.21   | 0.01   | 7.29    | 6.12   | 0.01   | 850.20   | 1239.5 | 0.02   |
| T.Node(ms) | 5.76    | 7.62   | 6.28   | 11.81   | 13.68  | 8.18   | 21.18    | 35.80  | 22.66  |
| Nodes      | 13      | 27     | 1      | 617     | 447    | 1      | 40135    | 34623  | 1      |
| Branches   | 6       | 13     | 0      | 308     | 223    | 0      | 20067    | 17311  | 0      |
| P.Rate(%)  | 0%      | 11.1%  | 100%   | 14.6%   | 17.7%  | 100%   | 21.7%    | 21.1%  | 100%   |
| Inf.       | 6       | 9      | N/A    | 218     | 144    | N/A    | 11342    | 10003  | N/A    |
| Int.       | 1       | 2      | N/A    | 1       | 1      | N/A    | 4        | 3      | N/A    |

**Điểm đặc biệt R101:** B&C chậm hơn B&B tại  $n = 11$  do overhead sinh ràng buộc cắt t

# R101 — So sánh thời gian chạy



\* B&C chậm hơn B&B tại  $n = 11$  do overhead sinh ràng buộc cắt

# RC101 — Hỗn hợp: Kết quả tổng hợp

| Tiêu chí   | $n = 5$ |       |       | $n = 8$ |        |        | $n = 11$ |        |        |
|------------|---------|-------|-------|---------|--------|--------|----------|--------|--------|
|            | B&B     | B&C   | B&P   | B&B     | B&C    | B&P    | B&B      | B&C    | B&P    |
| Obj        | 89.13   | 89.13 | 89.13 | 115.74  | 115.74 | 115.74 | 191.28   | 191.28 | 191.28 |
| Gap (%)    | 0%      | 0%    | 0%    | 0%      | 0%     | 0%     | 0%       | 0%     | 0%     |
| Time (s)   | 0.27    | 0.19  | 0.01  | 19.27   | 23.50  | 0.16   | 981.38   | 140.98 | 0.13   |
| T.Node(ms) | 5.57    | 6.41  | 12.02 | 15.18   | 18.20  | 162.76 | 21.49    | 30.23  | 131.98 |
| Nodes      | 49      | 29    | 1     | 1269    | 1291   | 1      | 45667    | 4663   | 1      |
| Branches   | 24      | 14    | 0     | 634     | 645    | 0      | 22833    | 2331   | 0      |
| P.Rate(%)  | 14.3%   | 10.3% | 100%  | 13.3%   | 14.7%  | 100%   | 17.8%    | 20.7%  | 100%   |
| Inf.       | 14      | 9     | N/A   | 462     | 447    | N/A    | 14712    | 1355   | N/A    |
| Int.       | 4       | 3     | N/A   | 4       | 9      | N/A    | 5        | 12     | N/A    |

**RC101 ( $n = 11$ ):** B&C giảm 89.8% số nút so với B&B (4663 vs 45667 nút)

# RC101 — So sánh thời gian chạy



\* B&P hoàn thành trong 0.13s so với 981s của B&B

# So sánh ảnh hưởng của phân bố dữ liệu

## C101 — Phân cụm

- B&C **hiệu quả cao**: giảm 88.8% số nút so với B&B ( $n = 11$ )
- B&P: chỉ 0.50s vs 473.92s
- Cận dưới B&B chặt  $\approx 50\%$  P.Rate

## R101 — Ngẫu nhiên

- B&C **tệ hơn B&B** ở  $n = 11$ : 1239s vs 850s
- P.Rate thấp: chỉ 21%
- Overhead sinh cuts > lợi ích
- B&P: 0.02s

## RC101 — Hỗn hợp

- B&C khá hiệu quả: giảm 89.8% số nút
- B&B lớn nhất: 45.667 nút
- 14.712 nút vô nghiệm (B&B)
- B&P: 0.13s

**Kết luận quan trọng:** Hiệu quả của B&C phụ thuộc mạnh vào **cấu trúc phân bố dữ liệu**. Trên dữ liệu có quy luật (C, RC), cuts giúp rút gọn cây đáng kể. Trên dữ liệu ngẫu nhiên (R), overhead vượt lợi ích.

# Bảng tổng kết ba thuật toán

| Tiêu chí                   | B&B             | B&C            | B&P           |
|----------------------------|-----------------|----------------|---------------|
| Gap                        | Tối ưu          | Tối ưu         | Tối ưu        |
| Thời gian ( $n = 11$ )     | 473s – 981s     | 78s – 1239s    | 0.02s – 0.50s |
| Số nút ( $n = 11$ )        | 20.521 – 45.667 | 2.305 – 34.623 | 1 nút         |
| P.Rate                     | 18% – 50%       | 20% – 50%      | 100%          |
| T.Node                     | 21 – 23 ms      | 34 – 36 ms     | 22 – 500 ms   |
| Dữ liệu Phân cụm (C)       | Kém             | Tốt            | Tốt nhất      |
| Dữ liệu Ngẫu nhiên (R)     | Trung bình      | Kém hơn B&B    | Tốt nhất      |
| Dữ liệu Hỗn hợp (RC)       | Kém             | Tốt            | Tốt nhất      |
| Nút vô nghiệm ( $n = 11$ ) | 7.122 – 14.712  | 864 – 10.003   | Không có      |
| Quy mô mở rộng             | $n \leq 11$     | $n \leq 11$    | $n \leq 100$  |

## 8.KẾT LUẬN

# Kết luận

Tất cả ba thuật toán đều đạt nghiệm tối ưu (Gap = 0.00%)

Tuy nhiên, chi phí tính toán để đạt được nghiệm có sự khác biệt rất lớn.

## B&P Branch & Price

- **Nhanh nhất** mọi cấu hình
- Chỉ cần **1 nút**: Column Generation tạo nghiệm nguyên tại gốc
- Mở rộng đến  $n = 100$
- T.Node cao do pricing subproblem

## B&C Branch & Cut

- **Tốt trên C, RC**: cuts hiệu quả
- **Kém trên R**: overhead > lợi ích
- Phụ thuộc mạnh vào phân bố dữ liệu
- Giới hạn:  $n \leq 11$

## B&B Branch & Bound

- **Chậm nhất** khi  $n$  tăng
- Cây tìm kiếm bùng nổ: 45.667 nút tại RC101
- Cận dưới không đủ chặt
- Giới hạn:  $n \leq 11$

CẢM ƠN QUÝ VỊ ĐÃ LẮNG  
NGHE!

Q&A